Elasticsearch - Cheat Sheet

Elasticsearch is a distributed search and analytics engine built on Apache Lucene. You write JSON documents to it, and it builds inverted indexes so full-text search runs in milliseconds even over terabytes. Queries are JSON. Results are JSON. The cluster handles sharding, replication, and failover.

The mental model is fast search over denormalized JSON. Populate it from your real source of truth. Use a relational DB for transactions and heavy updates.

Cluster, node, index, shard

The hierarchy and what each piece does.

Term	What it is
Cluster	A group of nodes that work together. Identified by `cluster.name`.
Node	A single Elasticsearch process. Roles: master, data, ingest, coordinating.
Index	A logical collection of documents. Like a “table” but really a virtual layer over shards.
Shard	A Lucene index. Each Elasticsearch index is split into one or more shards.
Replica	A copy of a shard on a different node. For redundancy and read throughput.
Document	A JSON object. The unit of indexing and search.
Field	A key inside a document. Has a type (text, keyword, integer, date, etc.).

A shard is the smallest unit of work. You scale out by adding shards across nodes, not by making one shard bigger.

The inverted index

The data structure behind every full-text search. Instead of “given a document, find its words”, an inverted index stores “given a word, find the documents”.

```
"the quick brown fox jumps over the lazy dog"

Inverted index:
  brown -> [doc1]
  dog   -> [doc1]
  fox   -> [doc1]
  jumps -> [doc1]
  lazy  -> [doc1]
  over  -> [doc1]
  quick -> [doc1]
  the   -> [doc1]
```

For “fox AND lazy”, look up each term and intersect the posting lists. Constant work per term, regardless of corpus size.

Every text field gets tokenized into terms, normalized (lowercased, stemmed), then stored. The analyzer decides how.

Documents and mappings

A document is a JSON object indexed under an index name and a unique ID.

```
PUT /users/_doc/42
{
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "age": 30,
    "tags": ["premium", "us"],
    "created_at": "2025-01-01T12:00:00Z"
}
```

A mapping is the schema for an index. Defines field types and how they’re analyzed.

```
PUT /users
{
    "mappings": {
        "properties": {
            "name":       { "type": "text" },
            "email":      { "type": "keyword" },
            "age":        { "type": "integer" },
            "tags":       { "type": "keyword" },
            "created_at": { "type": "date" }
        }
    }
}
```

Elasticsearch auto-detects a mapping if you index without defining one. Don't rely on this in production. Define mappings explicitly.

Common field types

Type	Used for
`text`	Full-text search. Tokenized, analyzed. Not aggregatable by default.
`keyword`	Exact match, sorting, aggregations. Not tokenized.
`integer`, `long`, `float`, `double`	Numbers
`date`	Dates with format support
`boolean`	true / false
`nested`	Array of objects, queryable independently
`object`	Plain JSON object (default for sub-fields)
`geo_point`, `geo_shape`	Geo data
`ip`	IPv4 / IPv6

The `text` and `keyword` distinction trips up most newcomers. `text` is analyzed (lowercased, tokenized) for search. `keyword` is stored as-is for filtering, sorting, and aggregation. Often a field gets both via multi-fields:

```
"email": {
    "type": "keyword",
    "fields": {
        "text": { "type": "text" }
    }
}
```

Query the analyzed form at `email.text` and filter or aggregate on `email`.

Analyzers

An analyzer transforms text into searchable terms. Three stages:

1. **Char filters** (e.g., strip HTML tags).
2. **Tokenizer** (split on whitespace, punctuation, language rules).

3. **Token filters** (lowercase, remove stop words, stem, synonym expansion).

The built-in `standard` analyzer covers most English text. For other languages or domain-specific tokenization, configure custom analyzers.

```
PUT /products
{
    "settings": {
        "analysis": {
            "analyzer": {
                "lowercase_only": {
                    "tokenizer": "standard",
                    "filter": ["lowercase"]
                }
            }
        }
    },
    "mappings": {
        "properties": {
            "name": { "type": "text", "analyzer": "lowercase_only" }
        }
    }
}
```

Indexing data

Single document

```
POST /users/_doc/42
{ "name": "Alice", "email": "alice@example.com" }
```

ID 42 explicit. Use `POST /users/_doc` with no ID to auto-generate.

Update

```
POST /users/_update/42
{
    "doc": { "age": 31 }
}
```

Updates aren't in-place. Elasticsearch reads the document, applies changes, and reindexes. Cheap-looking, expensive at scale.

Bulk API

For more than a few hundred documents, use bulk:

```
POST /_bulk
{ "index": { "_index": "users", "_id": "1" } }
{ "name": "Alice", "email": "alice@example.com" }
{ "index": { "_index": "users", "_id": "2" } }
{ "name": "Bob", "email": "bob@example.com" }
```

Each row is a JSON object on its own line. Action and doc pairs. Aim for 5-15 MB per bulk request.

Refresh and consistency

After indexing, the document isn't immediately searchable. The refresh interval (default 1 second) controls when new docs become visible to search.

```
POST /users/_doc/42?refresh=wait_for
```

Three values:

- `refresh=false` (default): doc becomes searchable on the next refresh cycle.
- `refresh=wait_for`: blocks until the next refresh cycle. Safer for read-your-writes.
- `refresh=true`: forces an immediate refresh. Expensive. Avoid in high-throughput paths.

This near-real-time model is Elasticsearch's tradeoff for write throughput.

Query DSL

All searches are JSON sent to `_search`. The query DSL has many query types. Here are the ones you'll write most.

Match query (full-text)

```
GET /users/_search
{
    "query": {
        "match": { "name": "alice johnson" }
    }
}
```

Tokenizes the query string ("alice", "johnson"), looks up each in the inverted index, scores by relevance. Use for human-friendly search.

Term query (exact match)

```
GET /users/_search
{
    "query": {
        "term": { "email": "alice@example.com" }
    }
}
```

No analysis. Looks for the exact term in the inverted index. Use on `keyword` fields. Using `term` on a `text` field rarely works because the term you wrote isn't what the analyzer stored.

Range

```
GET /users/_search
{
    "query": {
        "range": {
            "age": { "gte": 25, "lt": 40 }
        }
    }
}
```

Bool: combining queries

`bool` is the workhorse for compound queries.

```
GET /users/_search
{
    "query": {
        "bool": {
            "must":     [ { "match": { "name": "alice" } } ],
            "filter":   [ { "term":  { "status": "active" } } ],
            "must_not": [ { "term":  { "role": "banned" } } ],
            "should":   [ { "match": { "tags": "premium" } } ]
        }
    }
}
```

- `must` : matches required, contributes to score.
- `filter` : matches required, doesn't contribute to score. Cached.
- `must_not` : matches forbidden.
- `should` : matches optional, contributes to score (acts like OR if no `must`).

The single biggest performance win: put non-scoring criteria (status, date ranges, IDs) into `filter` . Filters are cached and skip the scoring step entirely. `must` runs scoring on every hit, which costs CPU even when you only care about whether something matched.

Other useful queries

Query	Use
`multi_match`	Same query across many fields
`match_phrase`	Exact phrase match (terms in order)
`prefix`	Starts with X (slow, use `search_as_you_type` field instead)
`wildcard`	Pattern match (`alice*`). Slow on large indexes.
`fuzzy`	Edit-distance match (handles typos)
`exists`	Field is non-null
`ids`	Match by `_id`
`geo_distance`	Within X km of a point

Aggregations

Run alongside or instead of search results. Compute counts, sums, percentiles, group-bys.

Metric aggregations

Compute a single value over the matched documents.

```
GET /orders/_search
{
    "size": 0,
    "aggs": {
        "total_revenue": { "sum": { "field": "amount" } },
        "avg_order":     { "avg": { "field": "amount" } },
        "p95_latency":   { "percentiles": { "field": "latency_ms", "percents": [95] }
        }
    }
}
```

`size: 0` means “don’t return matching documents, only the aggregation result”.

Bucket aggregations

Group documents into buckets.

```
GET /orders/_search
{
    "size": 0,
    "aggs": {
        "by_status": {
            "terms": { "field": "status", "size": 10 }
        },
        "by_day": {
            "date_histogram": {
                "field": "created_at",
                "calendar_interval": "day"
            }
        }
    }
}
```

Nested aggregations

Aggregations compose. A bucket aggregation can contain metric or bucket sub-aggregations.

```
{
    "aggs": {
        "by_status": {
            "terms": { "field": "status" },
            "aggs": {
                "revenue": { "sum": { "field": "amount" } }
            }
        }
    }
}
```

Result: for each status bucket, the total revenue. SQL would say `GROUP BY status` with a `SUM(amount)`.

Pagination

Three approaches, picked by use case.

From / size (avoid past a few thousand)

```
{ "from": 100, "size": 20 }
```

Simple. Slow as `from` grows. Each shard must collect and sort all docs up to `from + size`. Hard limit at 10,000 by default (`index.max_result_window`).

search_after (recommended for deep pagination)

```
{
    "size": 20,
    "sort": [ { "created_at": "desc" }, { "_id": "asc" } ],
    "search_after": [ "2025-01-01T12:00:00Z", "user-42" ]
}
```

Cursor-style. Pass the sort values from the last hit of the previous page as `search_after`. No upper bound. Requires a stable sort on a unique field as the tie-breaker (`_id` works).

Scroll API (legacy for paging, use for snapshots)

Snapshot view of the index. Memory-intensive on the server. Useful for one-time exports. The Point in Time (PIT) API plus `search_after` is the modern replacement.

Shards and replicas

Primary shards

A logical index is split into primary shards. Each primary lives on one node.

- Number of primaries is fixed at index creation. To change it, reindex.
- More primaries means more parallelism for indexing and search.
- Too many primaries means overhead per shard and slow cluster state.

Sizing guideline: 20-50 GB per shard for typical workloads. A 1 TB index probably wants 20-50 primary shards.

Replica shards

Copies of primary shards on different nodes.

- Default: 1 replica per primary.
- Replicas serve reads, so doubling replicas roughly doubles read throughput.
- Replicas don't help indexing. Writes go to primary, then replicate.
- Replicas need disk and memory. Don't set it to 5 just because you can.

Routing

By default, document ID hashes to choose the primary shard:

```
shard = hash(_id) % number_of_primary_shards
```

Custom routing:

```
POST /users/_doc/42?routing=tenant-7
{ ... }
```

All docs with `routing=tenant-7` land on the same shard. Useful for multi-tenant isolation, with the caveat that one big tenant can skew load badly.

Index lifecycle management (ILM)

Time-series indexes (logs, events) follow a lifecycle. ILM automates phase transitions.

Phase	Typical actions
Hot	Receiving new docs. SSD-backed nodes.
Warm	Older docs, less frequent reads. Move to cheaper nodes. Force-merge.
Cold	Rarely accessed. Searchable snapshots from S3 / GCS.
Delete	Drop after N days.

Combined with rollover (a new index each day or when a size threshold hits), this is how Elasticsearch handles log retention at scale.

Java client

The old `RestHighLevelClient` is deprecated. Use the Elasticsearch Java API client (`co.elastic.clients:elasticsearch-java`).

```
<dependency>
    <groupId>co.elastic.clients</groupId>
    <artifactId>elasticsearch-java</artifactId>
    <version>8.x</version>
</dependency>
```

```java
RestClient restClient = RestClient.builder(
        new HttpHost("localhost", 9200, "http")).build();

ElasticsearchTransport transport = new RestClientTransport(
        restClient, new JacksonJsonpMapper());

ElasticsearchClient esClient = new ElasticsearchClient(transport);

// Index a document
User user = new User(42L, "alice", "alice@example.com");
IndexResponse indexResp = esClient.index(i -> i
    .index("users")
    .id(user.id().toString())
    .document(user));

// Search
SearchResponse<User> resp = esClient.search(s -> s
    .index("users")
    .query(q -> q
        .match(m -> m.field("name").query("alice"))
    ),
    User.class);

for (Hit<User> hit : resp.hits().hits()) {
    User u = hit.source();
    System.out.println(u.email());
}
```

The fluent builder mirrors the JSON DSL. Type-safe at compile time.

Performance tips

- **Move non-scoring criteria into `filter`.** Filters are cached and skip scoring.
- **Pre-define mappings.** Dynamic mapping is fine in dev, dangerous in prod. A typo in a field name can balloon the mapping.
- **Avoid wildcards and `script` queries on hot paths.** They scan all documents.
- **Cap pagination depth.** Use `search_after` past page 50.
- **Bulk index.** Single-doc indexing is roughly 100x slower than bulk.
- **Tune `refresh_interval`.** During bulk loads, set to `-1` (no auto refresh) and force one refresh at the end.
- **Set `number_of_replicas` to 0 during initial bulk load**, then raise it after the load is done.
- **Skip `_source` when you don't need the full document.** Stored fields are heavy. `docvalue_fields` is cheaper for filtering and aggregation.
- **Use index aliases.** Apps point at the alias, ops swap the underlying index when rebuilding.

ELK and OpenSearch

The Elastic Stack (ELK):

- **Elasticsearch**: the search engine.
- **Logstash**: ingestion pipeline. Heavy.
- **Kibana**: visualization and admin UI.
- **Beats**: lightweight shippers (Filebeat, Metricbeat).

In 2021 Elastic changed Elasticsearch's license from Apache 2.0 to SSPL. AWS forked the project as **OpenSearch**. OpenSearch is API-compatible at the 7.x level but has diverged since. Pick based on your cloud provider and licensing tolerance.

Best practices

- Define mappings explicitly. Skip dynamic mapping in production.
- Use multi-fields to get both `text` (search) and `keyword` (filter, sort, aggregate) on the same data.
- Use `filter` clauses for any criterion that doesn't need scoring.
- Bulk index. Aim for 5-15 MB per bulk request.
- Plan shard count at index creation. Reindexing later is painful.
- Set `refresh_interval` based on requirements. 1s is the default. Some workloads can go to 30s for higher throughput.
- Use index aliases for indirection. The alias is what apps query. Indexes can be swapped behind it.
- For time-series data, use ILM + rollover. Don't keep a single growing index forever.
- Capture cluster health (`/_cluster/health`) in monitoring. Yellow means missing replicas, red means missing primary.
- Snapshot before risky changes. Restore is straightforward. Rebuilding from source is not.

Common gotchas

- **`text` vs `keyword` confusion.** Aggregating on a `text` field fails. Filtering with `term` on a `text` field returns nothing because the term was analyzed. Use multi-fields.
- **Mapping explosion.** Dynamic mapping with arbitrary keys (e.g., logging user-supplied JSON) creates thousands of fields. The cluster slows, then breaks. Use `dynamic: "strict"` or templates.
- **Refresh interval surprises.** A doc indexed at T+0 isn't searchable until T+1s by default. Tests that index then search immediately need `refresh=wait_for` .
- **Bulk request too large.** Past 100 MB the cluster rejects with 429. Aim for 5-15 MB.
- **Deep pagination death.** `from + size` past 10,000 fails by default. Use `search_after` .

- **One huge index.** A 1 TB index in 5 primary shards has 200 GB shards. Lucene struggles past 50 GB.
- **Replicas don't speed up writes.** They duplicate the work. Adjust them for read throughput and redundancy only.
- **`_source` disabled.** If you turn off `_source`, you can't reindex from the current index. You'd have to rebuild from upstream.
- **Tokenizer mismatch between index and query.** Indexed with `standard` analyzer, queried with a different one. Results don't match what you expect. Test with `_analyze`.
- **Wildcard at the start of a term (`*foo`).** Forces a full scan of the term dictionary. Very slow. Use a `reverse` token filter or change the data shape.
- **`script` queries everywhere.** Flexible but slow. Cache busts.
- **Forgetting tie-breaker in `search_after`.** Sort on a non-unique field without an `_id` tie-breaker can skip or duplicate results.

Quick reference

Concept	Key fact
Cluster / node / index / shard	Cluster of nodes, indexes split into shards
Inverted index	Word -> docs that contain it
`text` vs `keyword`	Analyzed for search vs exact for filter/sort/aggregate
Multi-field	One source, multiple indexed forms
`match` query	Tokenized full-text search
`term` query	Exact term lookup (use on `keyword`)
`bool` `filter`	Required, no scoring, cached. The fast path.
Bulk API	Indexing in batches of 5-15 MB
`refresh=wait_for`	Block until doc is searchable
`search_after`	Cursor-style deep pagination
Primary shard	Logical unit of storage. Fixed at index creation.
Replica	Copy of a primary on a different node
Routing	Pick the primary shard by a custom key
ILM	Hot / warm / cold / delete phase automation
Java client	`co.elastic.clients:elasticsearch-java` (RestHighLevelClient is deprecated)
OpenSearch	AWS fork after Elastic's license change